# Module 8: Context, Pipeline and Expert Parallelism

## *Splitting Context, Depth, and Experts*

**BRAIN team**

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

# Lecture map

1. Context parallelism
2. Pipeline parallelism
3. Expert parallelism
4. Composing parallel dimensions

**Guiding question**

When width sharding is not enough, how do we divide long contexts and deep models without losing exactness or utilization?

# What should be clear by the end?

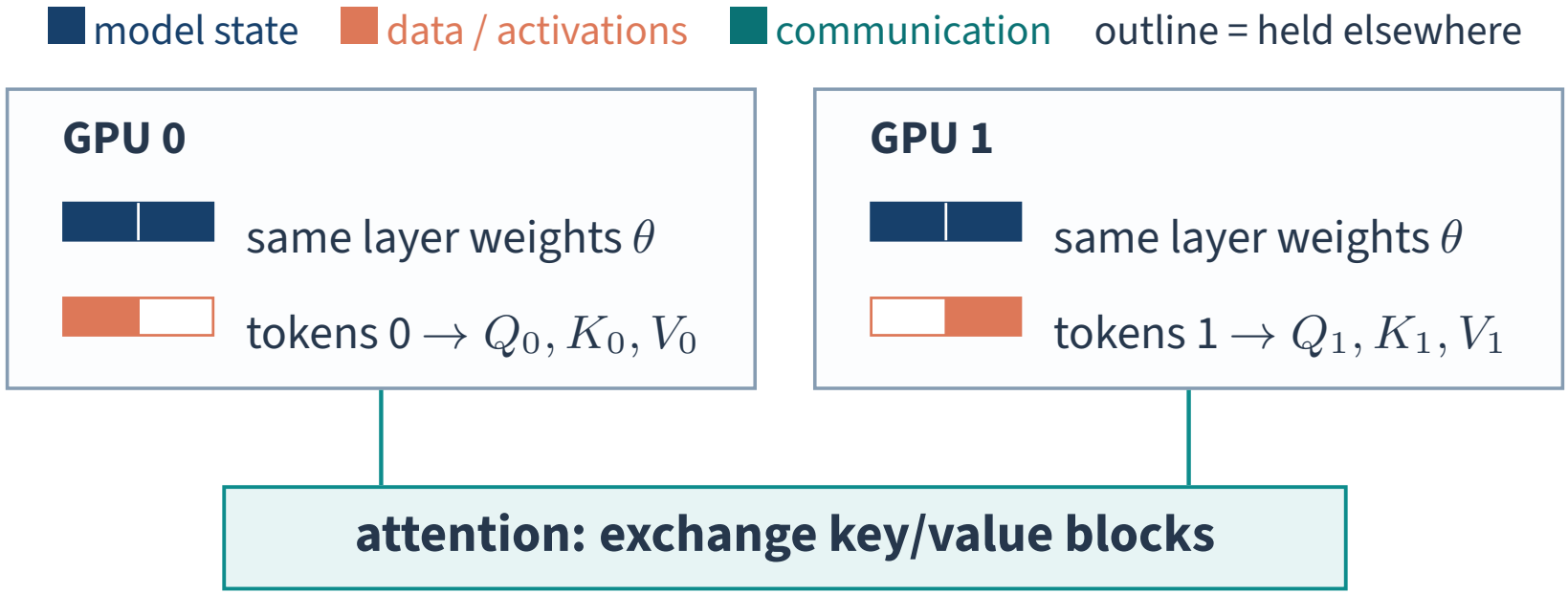
1. Distinguish context parallelism from sequence parallelism.
2. Explain exact ring attention through online softmax.
3. Trace pipeline schedules, token routing, and their communication.
4. Map parallel dimensions to a real cluster topology.

PART 1

# Context parallelism

1. Context parallelism
2. Pipeline parallelism
3. Expert parallelism
4. Composing parallel dimensions

# Parallelism map: CP splits the context




**What changes?** Each GPU owns different positions of one sequence. Queries need visible keys and values from other GPUs.

Causality masks future tokens; weight gradients also synchronize across context ranks.

# Two techniques shard the sequence axis for different reasons

### Sequence parallelism

Complements tensor parallelism by sharding token activations around norms, dropout, and elementwise operations.

### Context parallelism

Splits a long context across devices so each rank stores only part of the queries and token activations.

### The hard part

Attention couples every local query with keys and values that may live on every other rank.

# Most layers are local after token sharding

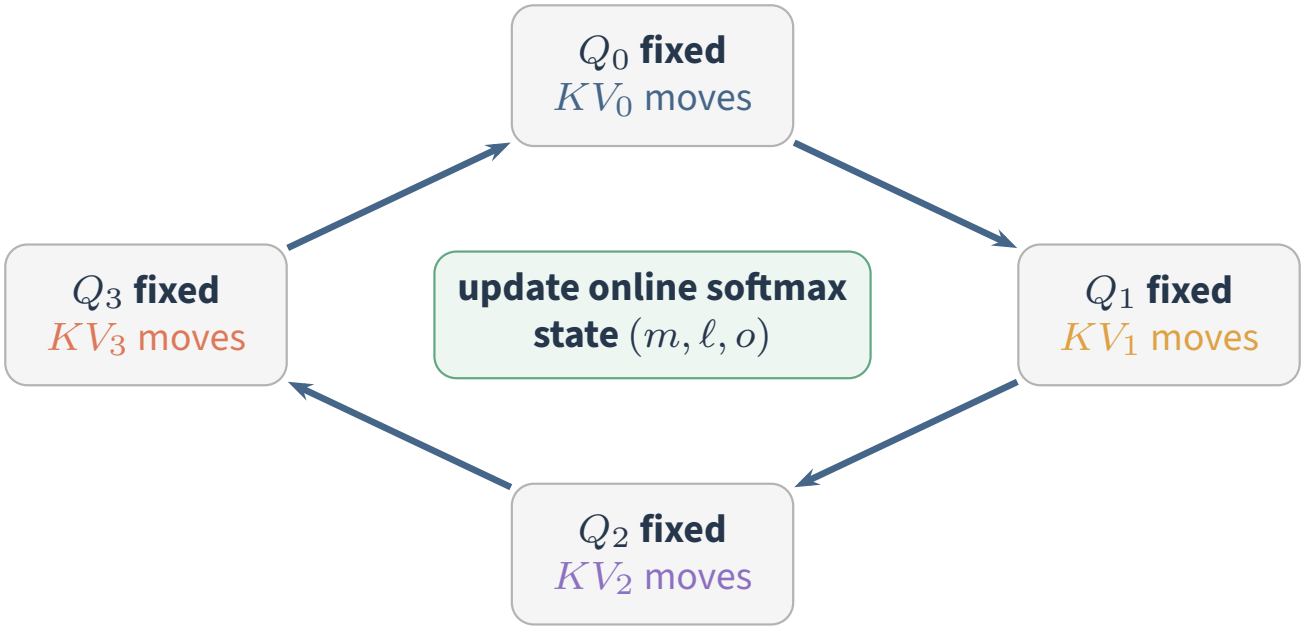
With $T$ split over $C$ context ranks, each rank owns roughly $T/C$ positions.

- tokenwise normalization and MLPs operate locally
- residual additions operate locally
- attention needs remote key-value blocks
- some losses and metrics need global reductions

**Memory effect**

Token activation memory falls roughly with $1/C$, while communication appears inside every attention layer.

# Ring attention rotates key-value blocks




**after $C$ rounds: every $Q_r$ has seen all $KV$ blocks**

## Memory behavior

Only a few $KV$ blocks are resident per rank; no rank gathers the full context or materializes a global $T \times T$ score matrix.

Source: Hugging Face Ultra-Scale Playbook and Ring Attention

# Online softmax makes blockwise attention exact

For one query row, maintain running maximum $m$, normalizer $\ell$, and numerator $o$. For a new score block with maximum $m_b$:

$$m' = \max(m, m_b)$$

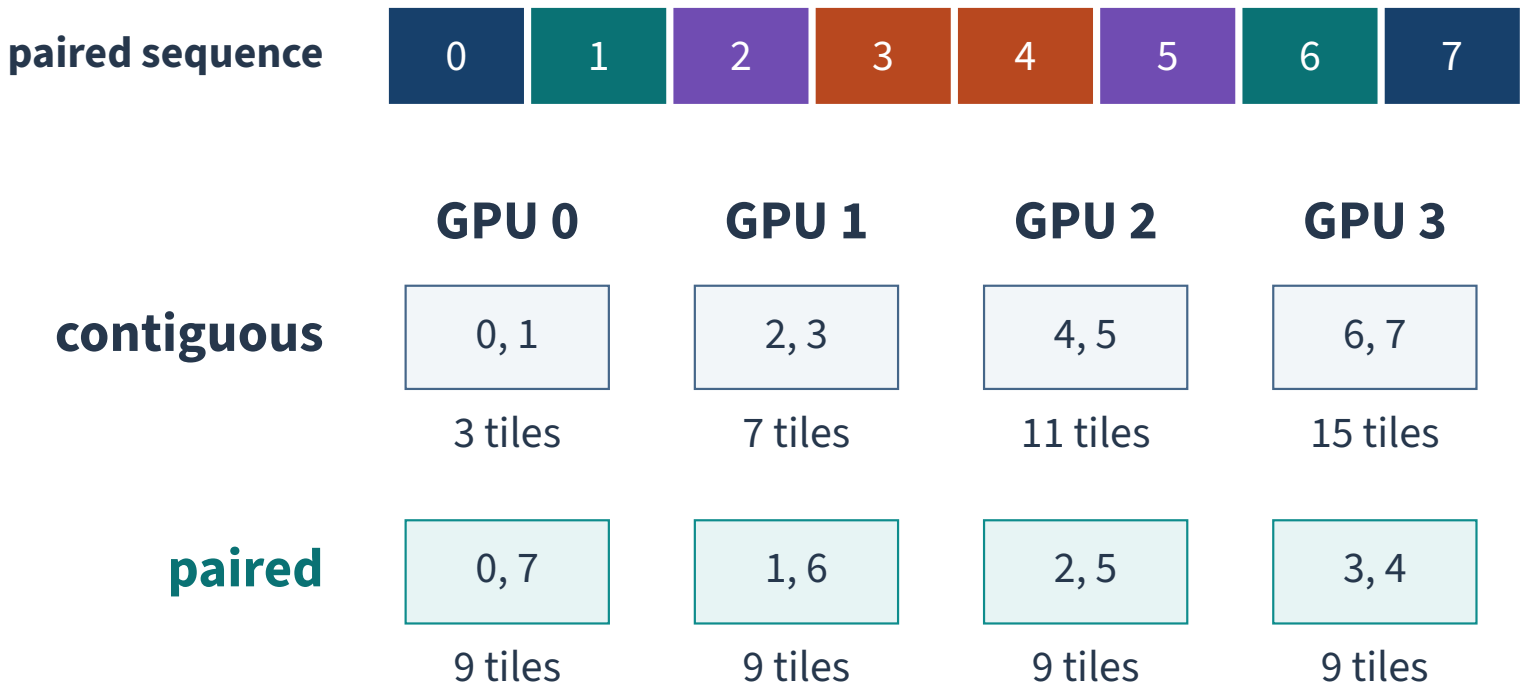
$$\ell' = e^{m-m'}\ell + \sum_j e^{s_j - m'}$$

$$o' = e^{m-m'}o + \sum_j e^{s_j - m'} v_j$$

Final attention is $o/\ell$. Rescaling preserves contributions computed under earlier maxima.

# Balance causal work by pairing early and late chunks

Split a sequence into eight equal chunks; count valid causal attention tiles per query chunk as $1, 2, \ldots, 8$.




Tile count includes each diagonal tile once; it is an illustration, not a runtime prediction. Pairing balances useful work but adds layout/scheduling considerations.

# CP communication: stream KV, then synchronize gradients

Let $C$ ranks split $T$ tokens; each local KV block contains $B(T/C)d_{kv}$ elements per K or V.

## Illustrative forward ring payload per rank

$$(C-1) \times 2B(T/C)d_{kv}\, b \quad \text{bytes sent}$$

$b$ is bytes/element; $d_{kv}$ is total KV width. Assumes $C-1$ full-block sends and equal chunks.

- Causal skipping, GQA, block layout and implementation change actual traffic and compute.
- Backward exchanges information for gradients; replicated weights require gradient synchronization across CP ranks.
- Overlap can hide some transfer, but exposed cost depends on link bandwidth and latency.

# Four parallelism maps, one comparison

| Scheme | Work split | Weights / state | Communication |
|---|---|---|---|
| DDP | different batches | full replica | average gradients |
| FSDP | different batches | persistent state shards | gather parameters; scatter gradients |
| TP | same tokens, different features | weight slices | combine activation slices / partials |
| CP | different positions, same sequence | full weights along CP axis | exchange attention K/V; combine weight gradients |

**Read this as a map, not four exclusive choices**

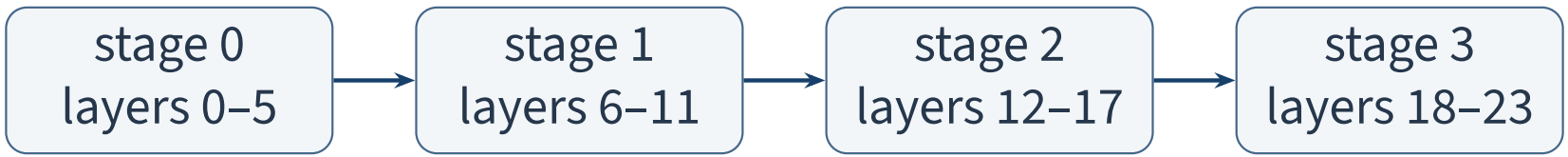
Each row isolates one parallelism axis. Real systems can compose several axes across the same cluster.

PART 2

# Pipeline parallelism

1. Context parallelism
2. Pipeline parallelism
3. Expert parallelism
4. Composing parallel dimensions

# Pipeline parallelism assigns layer ranges to stages




**Why micro-batches?**

Split a global batch into $M$ micro-batches so different stages can work on different micro-batches concurrently.

# Pipeline bubbles shrink as micro-batches increase

For $P$ balanced stages, $M$ micro-batches, and negligible communication:

$$\text{ideal utilization} \approx \frac{M}{M + P - 1}$$

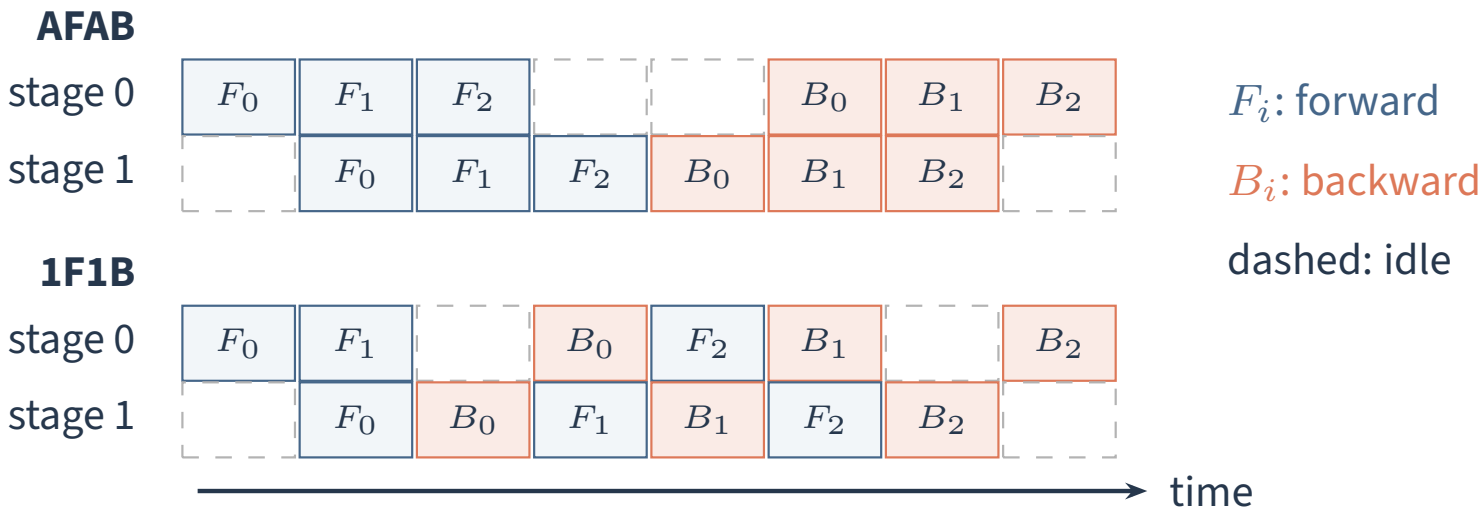
**Worked example**

$P = 4$, $M = 16$ gives idle fraction $3/19 \approx 15.8\%$.
Below $10\%$ requires $M > 9(P - 1)$, hence at least 28 micro-batches here.

**Trade-off**

More micro-batches reduce bubbles but can create smaller, less efficient kernels and more activation bookkeeping.

# Two stages, three micro-batches: compare schedules




Equal-duration slots, zero transfer time, one optimizer update. Same ideal makespan; 1F1B releases activations earlier.

# Explore pipeline schedules interactively

**Pipeline parallelism simulator**

**Open the pipeline schedule demo** ↗

1. Fix stages and micro-batches; compare AFAB with 1F1B.
2. Track stage idle time, boundary sends and outstanding activations.
3. Increase micro-batches; then try interleaving and compare added transfers.

Illustrative schedule model: real stage timings, communication, and memory must be measured.

# AFAB and 1F1B trade memory for schedule complexity

**All-forward-all-backward**

simple schedule
many outstanding activations
higher peak memory

**One-forward-one-backward**

after warmup, alternate directions
fewer stored activations
more coordination

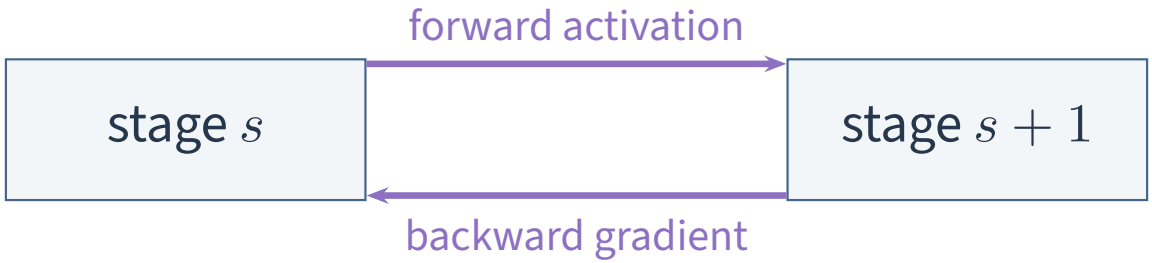
Both still have warmup and cooldown bubbles. Interleaving virtual stages can reduce them further at added complexity.

# The slowest stage sets pipeline throughput

$$t_{\text{cycle}} \approx \max_s t_s$$

- Equal layer counts need not mean equal work.
- Embeddings, output projection, and activation sizes can skew stages.
- Stage boundaries determine activation communication volume.
- Profile stage time and memory before moving boundaries.

# PP communication: activation and gradient at each boundary




For boundary shape $B_\mu \times T \times d$ and $b$ bytes/element: one forward activation and one backward gradient per micro-batch, approximately $2B_\mu Tdb$ bytes sent across the boundary.

**Cost depends on placement**

Latency, link bandwidth, stage imbalance and overlap determine exposed time. More micro-batches increase message count; total bytes depend on batch and schedule.

# Checkpoint: schedule arithmetic

**Think, then compare**

For six stages and twelve micro-batches, estimate idealized forward utilization. Name two real effects that make measured utilization lower.

PART 3

# Expert parallelism

1. Context parallelism
2. Pipeline parallelism
3. Expert parallelism
4. Composing parallel dimensions

# From one dense MLP to routed experts

**Dense MLP**

Every token passes through the same MLP.

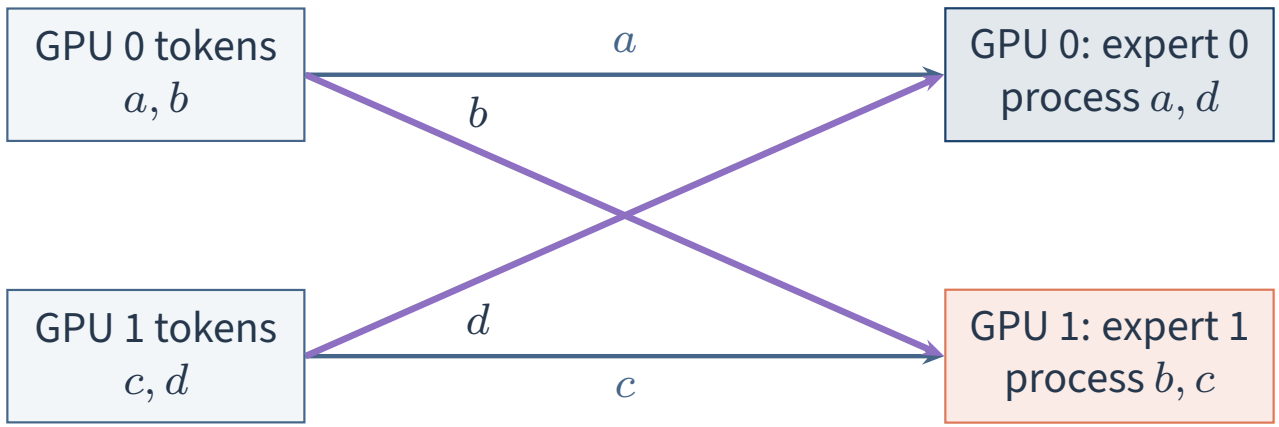
**Mixture of experts**

A router scores $E$ expert MLPs and selects top-$k$ for each token. Combine the selected outputs using routing weights.

$$y(x) = \sum_{e \in \text{top-}k(x)} p_e(x)\ \text{MLP}_e(x)$$

Total expert parameters grow with $E$; active expert work per token depends on $k$ and expert size. EP distributes expert weights across GPUs.

# Top-1 example: route tokens to expert MLPs




**purple: remote dispatch**

Each rank routes its tokens, exchanges remote assignments, and runs the experts it owns. Outputs return to token owners; the original order is restored.

Top-1 shown. Top-$k$ can send a token to several experts; self-routes stay local.

# EP uses all-to-all to dispatch and combine




## Why all-to-all?

Every source rank can send different token groups to every expert rank. Backward reverses the data movement and propagates gradients through selected experts and routing weights.

Group size, skew, message size and topology determine cost; implementations may use other dispatch primitives or fuse steps.

# EP communication grows with active routes

Let each rank originate $n$ tokens of width $d$, routed to $k$ experts; each element uses $b$ bytes.

## Simple forward payload per source rank

$$\text{dispatch + return} \approx 2nkdb \quad \text{bytes}$$

Counts one full vector per assignment in each direction. Uniform routing over $E_P$ ranks gives a remote fraction near $1 - 1/E_P$.

- Backward adds traffic; metadata, padding, skew and protocol overhead are excluded.
- Local routes do not traverse the network; coalescing and TP/SP layouts can alter payloads.
- Measure exposed dispatch/combine time and expert GEMM efficiency, not only average bytes.

# Load balance is an MoE performance constraint

### Skewed routing

One expert receives many tokens: its GPU becomes the straggler, buffers grow, and small experts run inefficient GEMMs.

### Ways to respond

Auxiliary balancing losses or router biases; capacity limits and optional padding/dropping; better dispatch and placement.

Monitor tokens per expert and rank, maximum / mean load, dropped tokens if applicable, and all-to-all time. Balancing can affect model behavior, not just throughput.

PART 4

# Composing parallel dimensions

1. Context parallelism
2. Pipeline parallelism
3. Expert parallelism
4. Composing parallel dimensions

# The device count factorizes across parallel groups

$$N_{\text{GPU}} = D \times T_p \times C \times P$$

| Dimension | Splits | Communication pattern |
|---|---|---|
| data | independent micro-batches | gradient or shard collectives |
| tensor | hidden width and heads | frequent activation collectives |
| context | sequence positions | key-value exchange per layer |
| pipeline | layer depth | point-to-point activations |

# Subgroups reuse existing devices

- $D$ counts the total data-parallel degree.
- FSDP degree $F$ divides $D$: $D/F$ groups replicate state shards.
- EP degree $E_P$ can divide $D$: each expert has $D/E_P$ replicas.
- Shared and expert parameters use different synchronization groups.

**Eight-device example**

$D = 4, T_p = 2, C = P = 1$. Setting $E_P = 4$ still uses eight devices.
Global tokens/update: $B_\mu\, T\, A\, D$.

Source: Teaching layout: combined EP/FSDP requires separate expert shard-group definitions

# Map the chattiest group to the fastest fabric

1. Start by placing frequent TP collectives on a fast fabric.
2. Add context parallelism when token activations or attention exceed local memory.
3. Use pipeline parallelism when model depth spans nodes.
4. Replicate the resulting model group with data parallelism when batch and devices remain.

**Selection rule**

State the rank layout and measure exposed transfer time. Persistent state excludes activations, gathered weights, and communication buffers.

# Four cases lead to different strategies

| Binding constraint | First strategy to evaluate |
|---|---|
| model fits, short context, more GPUs | DDP or FSDP replicas |
| one wide layer does not fit | tensor parallelism |
| weights fit, 128k-token examples do not | context parallelism plus checkpointing |
| model exceeds one node | pipeline across nodes, TP within node |

# Checkpoint: cluster design

**Think, then compare**

Design a layout for 32 GPUs arranged as four 8-GPU nodes. State the memory constraint, proposed $D, T_p, C, P$, and which communication crosses node boundaries.

# Key takeaways

1. Context parallelism shards tokens and uses exact blockwise attention with online softmax.
2. Pipeline parallelism shards depth, while micro-batches trade bubble size against memory and kernel efficiency.
3. Expert parallelism routes tokens through distributed MLPs using dispatch and return communication.
4. Count independent axes once and test placement against measured memory and communication.

# Explore a complete parallelism layout

**Parallelism composer**

**Open the parallelism composer** ↗

1. Choose model, devices and network topology; map TP, CP, PP and DP groups.
2. For a sparse model, add EP and inspect the expert placement and all-to-all estimate.
3. Compare memory, exposed communication and step-time estimates; change one choice and explain the result.

The composer is an analytical teaching model; check its assumptions and validate a real configuration with measurements. Combined EP/FSDP layouts are not modeled.